Cross-team Testing Interface

Goals

Main Idea: Team A should be able to run its own tests against Team B's system by changing the address it sends requests to.

Imagine your pod decides to build "What the Burger!", a system that classifies burgers from their ingredients. For this fictional project, everyone agrees on these rules.

- A beef patty without cheese → hamburger
- A beef patty with cheese → cheeseburger
- A vegetable patty, with or without cheese → veggie_burger

Every team builds two things:

- **system**: takes ingredients and returns a burger classification.
- **tester**: sends examples to someone's classifier and checks whether the answers are correct.

Suppose Team A writes a suite containing these two tests:

T-01: Beef patty with cheese → expect cheeseburger.
T-02: Vegetable patty without cheese → expect veggie_burger.

1. Set up a test session for Team B.

Send Team A's tester this target address:

```
{"target_url": "https://team-b.example.edu/system"}
```

A's tester creates run-123 (use a unique ID, such as a UUID, for each session) and passes it to B's system setup. B prepares the target and returns that ID when ready. One session covers one batch (suite): both tests share run-123. Keep the tester's ID, target URL, and report until the tester instance stops.

2. Run the whole batch once in this session.

Call A's POST /test/run once with {"session_id":"run-123"}.

In this example, A's tester sends two separate POST /classify requests to B in order, using run-123 for both:

```
{"session_id":"run-123","patty":"beef","cheese":true}
{"session_id":"run-123","patty":"vegetable","cheese":false}
```

B returns one response to each request, in the same order:

```
{"classification":"cheeseburger"}
{"classification":"veggie_burger"}
```

3. A's tester returns one report for the whole batch.

A stores and returns one suite report with both outcomes and coverage. Results excerpt (coverage omitted; see the complete format below):

```
{"session_id":"run-123","results":[
  {"test_id":"T-01","outcome":"pass",
   "message":"Expected cheeseburger; received cheeseburger."},
  {"test_id":"T-02","outcome":"pass",
   "message":"Expected veggie_burger; received veggie_burger."}
]}
```

4. To repeat the batch or test Team C, create a new session with a new ID and the chosen target URL.

OpenAPI and the testing interface

This document specifies both services that each team provides:

- **System**: Your project implementation. It prepares resources and handles requests through the pod's shared project API.
- **Tester**: Your automated test suite. It supplies inputs, keeps the expected outputs, and reports whether the system behaved correctly.

The pod defines correct behavior in a shared specification. Each team builds its own system and tester. Staff select a tester and a target system from the same pod; they may also test systems directly.

Across the course, every tester exposes /test/setup, /test/run, and /test/report. Every system exposes /test/setup and /test/report, plus its pod's project API. Within a pod, project paths, methods, inputs, responses, and behavior must match the shared specification. Teams choose their internal implementations. The tester owns the test sequence; a system /test/run endpoint is not required. Deploy both services and register their base URLs.

For example, A's tester setup is https://team-a.example.edu/tester/test/setup, and B's system setup is https://team-b.example.edu/system/test/setup. Tester setup passes its session ID to system setup and waits until the system is ready. You may connect these endpoints to an existing API.

Who calls what? An Example

Yongjae, a student from Team A, uses Team A's tester to test Team B's system:

1. Yongjae calls A's tester setup with B's system base URL in target_url.

2. A's tester generates a unique session ID and sends it with starting data to B's system setup. B prepares resources and returns the same ID when ready. A returns that ID only after preparation succeeds. If setup fails, A returns the failure to Yongjae and no tests begin.

3. Yongjae calls A's tester run. A executes its test sequence in its own code, calls B's regular project API, compares responses with expectations, and records the outcomes. After the project calls finish, it reads B's system report for code coverage and returns one suite report with results and coverage.

4. If the tester run response is lost or the tester aborts, retrieve its stored report using tester report with the session ID. This lookup does not start or repeat tests.

Required calls

Use JSON request and response bodies for the calls below. A base URL is the deployed address before the endpoint path. The pod specification defines project paths, methods, inputs, responses, expected errors, and how requests identify the state prepared by system setup.

Tester

`POST /test/setup`

Generates a unique session ID, retains it with the target URL, and passes the same ID with starting data to system setup. Waits for the system to be ready. Does not run tests.

Request body: `{"target_url":"https://team-b.example.edu/system"}`

Response: 201 with {"session_id":"run-123"} after system setup succeeds. If system setup returns an error, return the same status, message, and source. Failed setup does not create a runnable tester session.

`POST /test/run`

Executes its test sequence in tester code through the prepared target's regular project API. Evaluates responses and collects code coverage from system report. Stores and returns one complete report. Keeps available results if the tester aborts.

Request body: {"session_id":"run-123"}

Response:

- 200 with the complete suite report when the tester finishes, **including failed tests and reported system errors**.
- 500 with a message and source "tester" if the tester aborts. Retrieve its stored report through /test/report.

Example response:

```
{
  "session_id": "run-123",
  "target_url": "https://team-b.example.edu/system",
  "results": [
    {"test_id":"T-01","outcome":"pass",
     "message":"Expected cheeseburger; received cheeseburger."},
    {"test_id":"T-02","outcome":"fail",
     "message":"Expected veggie_burger; received hamburger."}
  ],
  "coverage": {
    "code": {
      "covered": null,
      "total": null,
      "basis": "Source lines; target instrumentation unavailable"
    },
    "component": {
      "covered": 2,
      "total": 3,
      "basis": "Components listed in the pod specification"
    },
    "system": {
      "covered": 3,
      "total": 5,
      "basis": "Acceptance criteria listed in the pod specification"
    }
  }
}
```

Include every planned test exactly once. Each result must have a unique, nonempty `test_id` and a nonempty `message`.

Use one of these outcomes:

- **pass:** The behavior matches expectations.
- **fail:** The response can be evaluated and differs from expectations.
- **error:** A tester, system, or communication problem prevents evaluation.
- **skip:** The test was not executed.

If the system returns the expected project error, mark the test pass.

For **error** results, also include source: `tester`, `system`, `communication`, or `unknown`. An unexpected internal failure reported by the target is an error with source: system. A timeout with no response is an error with source communication. Use unknown if the failure location cannot be identified.

For example, if a project API request times out and the tester receives no response to evaluate:

```
{
  "test_id": "T-03",
  "outcome": "error",
  "source": "communication",
  "message": "The /classify request timed out. No response was available to
evaluate."
}
```

Coverage fields

The coverage field in the response above describes the target system exercised by this suite. The tester combines all three measures with the test results in the same response.

- **Code coverage:** Source lines or branches executed. The system should measure this; the tester retrieves these measurements through system /test/report and includes them in its report. If measurements are unavailable, use `null` for both counts and explain why in `basis`.
- **Component coverage:** The tester computes how many pod-defined components its tests covered. The pod specification defines the component list and what counts as exercising each component.
- **System coverage:** The tester computes how many pod-defined acceptance criteria its tests covered. The pod specification defines the criteria and what counts as exercising each one.

Each entry contains `covered`, `total`, and a nonempty `basis` explaining the measure and its scope. Use nonnegative integer counts with `covered` no greater than `total`. For any unavailable measure, use `null` for both counts and explain why; do not substitute zero.

Important: coverage only pertains to what was covered by tests, while `results` records whether the system behaved correctly under test!

`GET /test/report?session_id=run-123`

Retrieves the stored suite report after a timeout, lost response, or abort. Does not start or repeat tests.

Request body: No body.

Response: 200 with the stored suite report, using the same format as a successful run response; 409 before the suite ends. After an abort, the report includes available results and identifies tests that could not be evaluated or executed.

System

The system implements the pod's project API. The tester owns test order and assertions, calls the regular project operations, and records suite results. No system batch endpoint or encoded test-sequence format is required.

`POST /test/setup`

Prepares resources and initial state for the supplied session ID. Returns the same ID when ready. Does not run tests.

Request body: {"session_id":"run-123","setup":{}}.

Use the tester's session ID, or a unique ID supplied by staff for direct testing; setup contains pod-defined starting data. Use {} if none is needed.

Response:

- 201 with {"session_id":"run-123"} when resources and initial state are ready.
- 409 if the session ID already exists.
- 500 with a nonempty message and source "system" if preparation fails. Example: {"message":"Could not initialize the session database.","source":"system"}. The tester returns this failure to its caller and does not begin the suite.

System setup is the prepare-for-test step. It may start resources or initialize data on demand. The setup endpoint must remain reachable even when those resources are stopped.

GET /test/report?session_id=run-123

Returns code coverage measured for this prepared session so far. The tester reads it after its project API calls finish and includes the measurements in the suite report. Does not execute project operations.

Request body: No body.

Response: 200 with the coverage response below; 409 while preparation is still in progress.

Example response:

```
{"session_id":"run-123","coverage":{"code":{
  "covered":null,"total":null,
  "basis":"Source lines; target instrumentation unavailable"
}}}
```

Include coverage.code using the covered, total, and basis fields defined in the tester section. Measurements cover the project requests made in this session since setup. Reading the report does not change them; later project requests may update them. The tester records test outcomes and computes component and system coverage.

Execution rules

- **Sessions:** The tester creates a unique, nonempty ID (such as a UUID) for each suite; never reuse it. The system uses and returns the supplied ID. Failed setup does not make the session ready for tests. Keep target state and coverage available through the suite and coverage retrieval. Keep tester reports until the tester instance stops; restarts may clear them. Teams choose how to track sessions and test instances internally. The pod documents preparation, resets, and cleanup behavior for cross-team testing.
- **Runs:** One suite per successfully prepared tester session. Reject concurrent or later tester runs with 409, including after an abort. Invalid requests do not consume a session. The tester executes its planned tests in order. The pod defines state changes or resets between project requests.
- **Tester reports:** Store the complete report before returning 200, or available results before returning 500 after an abort. Tester report lookup returns the stored body without changing state, or 409 before the suite ends. Keep the report unchanged once the suite ends. If the tester aborts, keep recorded outcomes, mark tests prevented by the abort error with source tester, and mark unexecuted tests skip. Target failures may appear in a normally completed tester report. A service crash may leave no retrievable report.
- **Errors**: For tester operations and system test endpoints, use 400 for invalid requests, 404 for unknown sessions, 409 for conflicts or reports not ready, and 500 for aborted operations or service failures. Error bodies require a nonempty message and source: tester, system, communication, or unknown. Preserve the source when forwarding a system setup error. If setup fails inside the tester, return 500 with source "tester"; if no system response is available because communication failed, use source "communication". The pod defines project API responses, including expected project errors.
- **Timing**: Setup waits for preparation. Tester run waits for the suite and returns its report. Register limits for tester setup/run and system setup; tester limits include all target calls. The pod defines project-call time limits and retry rules. After a tester run timeout or lost response, retrieve the tester report; if lookup returns 409, try it again later. Do not resend tester run. Use a new tester session to repeat the suite.

- **Session state and concurrency:** The pod decides whether sessions may run concurrently and whether tests share state or start from a reset baseline. Teams choose the implementation, but must prevent unintended interference between tests and sessions. For example, systems allowing only one active session at a time may return 409 for conflicting setup requests. Consider resets and cleanup behavior, including what happens after interruptions.
- **Access:** Use test data and approved pod service URLs. If authentication is required, document how graders and podmates obtain access, including the tester's access to the target. Use `401`/`403` where appropriate; do not put credentials in URLs or the shared spreadsheet.

Register your services

Fill your team's row in **the endpoint registry spreadsheet**:

Field	What to enter
System base URL	The actual deployed system address, for example `https://team-b.example.edu/system`.
Tester base URL	The actual deployed tester address, for example `https://team-b.example.edu/tester`.
Intra-pod specification URL	Link to the pod's shared project API, preparation, resets, cleanup, and acceptance criteria. Use the same specification within the pod.
Access / run notes	Access instructions and maximum times for tester setup/run and system setup. Tester times include target preparation, project calls, and coverage retrieval.

These are example addresses, not working services. Provide reachable HTTP(S) base URLs with no `/test` suffix, query, fragment, or embedded credentials. Tester and system URLs must be distinguishable; different paths on the same host are sufficient.

How we will grade your services

We will:

1. Select a tester and a target system from the same pod in the registry, including the team's own system.

2. Call tester setup with the target system's base URL. Wait for 201 and the returned session ID within the registered setup limit. If setup fails, inspect the message and source.

3. Call tester run with returned session ID and inspect the report in its response. If the response is lost or run returns 500, retrieve the stored tester report.

4. Inspect outcomes, coverage, and failure sources. For independent tests, staff supply a unique ID to system setup, call the project endpoints defined by the pod, and compare their responses with the pod specification. Read system report for code coverage.

To test another team or repeat a suite, create a new tester session with the selected target_url.

Phase 3 cross-team check

Run the tests at least once in both directions with at least one other team in your pod: your tester against their system, and their tester against yours. Record the teams, date, results, and problems to fix.

For Phase 3, the goal is to confirm that the common interface works and returns well-formed reports. Tests do not need to fully pass for full credit on this check.

Minimal OpenAPI starter for the tester

Use this OpenAPI starter to describe your tester's three endpoints. Replace the example URL with your tester's registered base URL, and follow the request and response formats shown above. If you use tools to validate messages or generate code from OpenAPI, add schemas that define the required fields.

Your system needs its own API description covering `/test/setup`, `/test/report`, and the project endpoints your pod agreed on.

```
openapi: 3.1.1
info:
  title: Team Tester Interface
  version: 0.4.2
servers:
  # Replace with the tester base URL.
  - url: https://team.example.edu/tester
paths:
  /test/setup:
    post:
      summary: Create a session and wait for system setup to succeed
      requestBody:
        required: true
        content:
          application/json:
            # Request: target_url.
            schema: {type: object}
      responses:
        '201':
          description: System ready; same session ID returned by tester and system
          content:
            application/json:
              schema:
                type: object
                required: [session_id]
                properties:
                  session_id: {type: string, minLength: 1}
        default:
          description: Setup failed; body contains message and source
          content: {application/json: {schema: {type: object}}}
  /test/run:
    post:
      summary: Run the tester sequence and return the complete report
      requestBody:
        required: true
        content:
          application/json:
            # Request: session_id.
            schema: {type: object}
      responses:
        '200':
          description: Complete tester suite report with results and coverage
          content: {application/json: {schema: {type: object}}}
  /test/report:
    get:
      summary: Retrieve the stored report without running tests
      parameters:
        - name: session_id
          in: query
          required: true
          schema: {type: string, minLength: 1}
      responses:
        '200':
          description: Stored tester suite report with results and coverage
          content:
            application/json:
              schema: {type: object}
        '409':
          description: Execution has not ended; retry the lookup later
```

References: Phase 2 testing interface · OpenAPI 3.1.1